Contraction trees and streamed Cauchy features for subexponential SAT

A review of the combined ProveIt frontier and two further improvements

Research analysis prepared for Vladimir Reshetnikov
with ChatGPT

October 26, 2026 (Pacific time)

Abstract

We review fifteen refinement packets through the closing snapshot `ProveIt@474ddba696a5d385c091c23fb2995f69edd13e00`. The initial snapshot contained seven placed packets and seven incoming archives; a final placement check added one matching-based packet without changing the frontier. Their ingredients compose: full-orientation Cauchy modes, asymmetric packet balancing, sharp Hermite precision, and cached mode-graph evaluation give a substantially stronger baseline than any standalone report indicates. We obtain two further results. First, constructing cached group tables by a binary contraction tree, and implementing fast matrix multiplication through exact integer-coordinate lifts, gives a relative contraction certificate

$$T(s, \mathcal{L}) \leq \mathrm{poly}(s + \mathcal{L}) 2^{(35.3545880584\ldots + o(1))s/\sqrt{\log_2 s}},$$

with space coefficient $34.2748030050\ldots$ on the same scale, under the common high-degree network interface. The proof pays for every recursive child and for exact arithmetic over the two-adic coefficient ring; it does not posit a free group oracle. Second, a constant-storage recurrence reduces the generation of B Cauchy features at P ports from $O(L + PB^2)$ to $O(L + PB)$ ring operations at unchanged precision. A matched modular implementation gives a measured 101.95-fold local speedup for one $B = 512$, 128-bit fixture, with identical outputs. The report gives complete proofs of the new components, exact finite checks, paired benchmark data, and integration notes. The general SAT logarithmic order is unchanged. The numerical constants count network wires, not SAT variables, and the asymptotic result retains the stated upstream contraction and reduction dependencies. No complete SAT solver or new Lean/Rocq certificate is claimed.

Deliverable scope. This is a research and implementation packet for review. The ZIP contains this PDF, its LaTeX, original Python components, all reported numerical results, checks, immutable source identifiers, and reproduction instructions. The package is designed to remain below 4 MB and does not duplicate the source manuscripts.

Contents

1 Results and the comparison that matters **3**
1.1 Two concrete advances . . . 3
1.2 What can and cannot be compared numerically . . . 3

2 Commit review and the fifteen-packet frontier **4**
2.1 Previously placed packets . . . 4
2.2 The eight second-round arrivals . . . 5
2.3 What is inherited and what is new . . . 6

3 The combined baseline and a replacement guarantee **6**
3.1 Reconstructing the existing 16-group cached bound . . . 7
3.2 Why the replacement preserves the matched bound . . . 7

4 A stronger cached contraction by recursive table construction **8**
4.1 The exact pair-index interface . . . 8
4.2 Fast matrix multiplication over the actual coefficient ring . . . 9
4.3 Build block tables recursively . . . 10
4.4 A rational schedule with strict margins . . . 11
4.5 Sum the entire recursion, including all child oracles . . . 12

5 Linear-work evaluation of full-crossing Cauchy features **13**

6 Executable evidence and reproducibility **14**
6.1 Exact checks . . . 14
6.2 Paired local benchmarks . . . 14
6.3 Reproduction commands . . . 15

7 SAT consequences, integration, and remaining questions **15**
7.1 Integration plan . . . 16
7.2 Further research questions . . . 16

8 Evidence ledger and limitations **17**

1 Results and the comparison that matters

The requested comparison is with an upper bound incorporating all the existing proposals. It is therefore insufficient to compare only with the original preprint or with the smallest number printed in an individual report. We explicitly compose compatible improvements and retain separate time–space regimes.

1.1 Two concrete advances

The new asymptotic component changes how the cached mode graph is evaluated. A cached function for a subset of groups has only the indices crossing that subset as arguments. Joining two disjoint subsets is a rectangular matrix multiplication: the shared internal indices are summed out, and their external indices form the output table. Constructing larger block tables this way avoids the earlier exhaustive enumeration of every incident index in a block. A fixed field matrix multiplication exponent below 12/5 is a classical available ingredient [9]; an explicit lifting argument makes it safe for the actual quotient rings used by the SAT framework.

The new implemented component reuses the prefix products in the full-orientation Cauchy basis. It preserves the mode weights, exact target/source features, and working precision. Its operation-count improvement is unconditional for the specified kernel, and it can be used in both polynomial-space streaming and cached contraction variants.

Certificate on the scale $s/\sqrt{\log_2 s}$	Time coefficient	Space
Strongest incoming standalone polynomial-space certificate	76.908995	polynomial
Asymmetric schedule plus joint Cauchy modes	66.605143	polynomial
Same supplied schedule plus existing cached Strassen method	44.537572	39.495950
New contraction trees, exact fast backend, explicit schedule	35.354588	34.274803

All entries refer to the common normalized high-degree tensor interface. The two middle rows are compositions derived in this review, not additional delivered packets. The third row is an explicit feasible combined baseline; it is not a claim that every parameter retuning of the prior ideas has been optimized. We give same-parameter comparisons and a general replacement inequality below so that the gain does not rest solely on that choice of baseline. Lower-degree specializations have different premises and are not ranked against the generic interface in this table.

The final row implies the simple eventual bounds

$$T \le \mathrm{poly}(s+\mathcal{L})2^{35.36s/\sqrt{\log_2 s}}, \qquad S \le \mathrm{poly}(s+\mathcal{L})2^{34.28s/\sqrt{\log_2 s}}$$

for all sufficiently large admissible inputs. The 35.36 and 34.28 are rounded *upward* after absorbing the vanishing errors. This improves an explicit exponent coefficient, not the power of the logarithm. An improvement of upper bounds alone is not a measured ratio of SAT running times.

1.2 What can and cannot be compared numerically

The size s is the number of Boolean network wires. A general-density SAT reduction uses $s = O_k(n \log\log n)$; density-sensitive variants use fewer wires in intermediate regimes. The constants

hidden in that reduction must be tracked before placing a numerical coefficient on a formula in n. In particular, 35.354588 must not be substituted for the hidden constant in the general SAT bound.

The separate manuscript credited to Claude states the same relevant logarithmic order with unspecified comparable constants [7]. No numerical dominance over that unspecified constant follows. Likewise, conditional proposals involving an unconstructed summation or compression algorithm cannot be treated as already implemented algorithms. Our result neither disproves those targets nor establishes them.

A safe portfolio statement is useful. Retain all admissible existing algorithms and add the new cached variant; choose using certified bounds, or dovetail the fixed collection. Up to a fixed overhead, its time is bounded by the minimum of their guarantees. Thus special lower-degree regimes and any separately superior construction remain available. The mathematical result here is a new explicit certificate and a cheaper compatible subroutine, rather than a claim of an optimal SAT algorithm.

2 Commit review and the fifteen-packet frontier

The initial review is pinned to the main-branch snapshot `ab2fbc9e03d5c3d0ad6dec8686960bfe498b0b06`; a closing check pins `474ddba696a5d385c091c23fb2995f69edd13e00`. The relevant path history was read through the GitHub connector, and the source bytes were independently retrieved from the public repository. Git blob hashes and SHA-256 digests were recorded for 138 files under the two requested paths and the vendored preprint directory. All seven incoming ZIPs were extracted into separate directories after checking their member paths; inspection preceded execution of any imported Python module.

The relevant recent commits are:

- `a505bd768b`, `64b3202ee8`, and `8afdbbea48`: arrivals of the earlier SAT/ETH reports and source PDFs.
- `2b925d8800`: placement of seven refinement packets under `Computability/SubexponentialSAT/Research`, plus the separate Claude manuscript.
- `3e09608991`: an unchanged vendored copy of the OpenAI manuscript directory.
- `45233238ae`: the project README's summary of the earlier seven packets.
- `3e57705cf1`: seven newer research archives in `docs/incoming`.
- `d098ba436b`: the additional matching-packing report.
- `6d3adc450615`: placement of all eight second-round packets; the closing main snapshot is `474ddba696a5`.

These commits fall on October 26 in Pacific time; the later Git timestamps are October 27 UTC. Initial commit messages and source URLs are in `provenance/review_manifest.json`; closing identities, placement paths, and byte checks are in `provenance/closing_provenance.json` and `audit/closing_review.txt`. The repository explicitly describes these as external research material; placement alone was not a proof verification or an executed test run [1].

2.1 Previously placed packets

Packet	Useful established component claimed there	Boundary
`cauchy_mode_improvement`	Factorial modes $1+b+2\nu_2(b!) = 3b+1-2s_2(b)$; entropy slope three.	Same logarithmic order; a kernel, not a SAT solver.
`eth_contraction_improvements`	Exact Hermite-condition count, precision $6MD/u$, coefficient 648.4411, and streamed packet extraction.	The coefficient is an upper-bound certificate.
`3sat_precision_refinement`	Exact recovery loss and the real full-loss minimax taper; leading precision six is optimal in that model.	Its $n \log\log n/\log n$ route needs a missing summation algorithm.
`sat_density_improvement`	Uniform density envelope with $\min\{\log(2 + m/n), \log\log n\}$.	No stronger general-density logarithmic order.
`eth_density_refinement`	Closely overlapping density result and two-adic Smith/precision observations.	Several comparisons improve constants only.
`3sat_refinement`	Scalar parity lift with incidence-degree cap $k(p+a-1)$.	Compact recursive mode factorization is not supplied.
`3sat_asymptotic_analysis`	Scoped barriers for taper, budget, and mode retuning; precise missing interfaces.	No stronger unrestricted exponent proved.

At the closing snapshot, all eight later packets are under `Research`, and `docs/incoming` contains only its README. We verified 312 Git blob matches with no mismatches: 120 original research files, 187 placed files from the seven initially reviewed archives, two updated indexes, and three texts from the newly arrived matching packet. Its report was read before closing the review. The placement map and exact checks are in `provenance/closing_provenance.json` [8].

The source directory is the same at the two upstream pins used by the older reports. The vendored reference pin is `openai/math@7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a` [3]. We retain the sharper manuscript appendix interface, while noting that the repository's linked Lean scope page states a weaker general bound. No formalization build was run.

2.2 The eight second-round arrivals

Packet stem	Principal addition	Time coefficient
`eth_asymmetric_packet_balancing`	Two-stage pair-load balance; fresh-side vs. retained-side degree invariant; additive precision buffer.	76.908995
`3sat_round2_refinement`	Matching-potential balance and different root/tail alphabets under a symmetric degree constraint.	84.066271
`eth_balanced_packing`	Near-unit padding and revised schedules.	94.748202
`eth_high_moment`	Higher-moment conditional expectations, thin precision guard, and a smaller initial degree exponent in a SAT reduction.	94.748202
`eth_matching_refinement`	Matching-potential near-unit padding and thin gaps; overlaps the same already covered coefficient.	94.748202
`eth_packet_refinement`	Exponential conditional potentials and improved packing conditions.	95.663940
`eth_recursive_refinement`	Missing-residue basis, including slope 64/21, and refined recursive scheduling.	133.207372

Packet stem	Principal addition	Time coefficient
`eth_joint_cauchy`	Full crossing-orientation factorization, slope four at every valuation; fully charged caching and Strassen evaluation.	319.685757

The last packet also gives a faster cached certificate 225.597057 with space coefficient 196.199446, using its older packing/schedule parameters. These numbers must be recomputed after composition; comparing our result only with 225.6 would miss most of the existing gain. Archive names, exact member hashes, and a detailed incoming audit are included in the package [2].

2.3 What is inherited and what is new

The following are inherited: the deformation $W(T)$, integral local coordinates, creation-precision convention, recovery identity, child-network factorization, sharp Hermite precision, full-orientation Cauchy weights, exact-weight index lists, asymmetric pair balancing, and the idea of caching group functions. Variable root/tail alphabets and geometric depth allocations also already occur in the corpus.

The new work is the binary construction of cached block tables; its complete use with an exact field-to-quotient-ring multiplication backend and a validated explicit schedule; and the proved and implemented streaming prefix evaluator. Classical matrix multiplication, CRT, dynamic programming, and prefix products are credited techniques. Novelty is claimed only for this refinement and integration relative to the reviewed corpus, not as a global priority claim for those techniques.

3 The combined baseline and a replacement guarantee

At an active call, let M be the wire count, $L = 2^a$ the packet length, i a retained valuation, $D = L/2^i$, r the grouping alphabet, and u the taper length. Near-unit padding has total mass $(1 + o(1))M$. The inherited exact recovery analysis gives

$$p_t \le (6 + o(1))\frac{MD}{u}.$$

For each directed pair of distinct groups there are at most $(2 + o(1))M/(r^2D)$ mode variables of each index q. The full-orientation weights are

$$w_q = a + 1 - i + 4q - 2s_2(q).$$

Every mode has exactly one source group and one target group, so replacing its basis does not add wires or enlarge packet-vertex degrees [5].

Partition mode tuples by the exact weight assigned to each directed pair. There are polynomially many such arrays because r is fixed and p_t is polynomial in s. Each pair's list can be indexed by the ordinary unbounded-knapsack recurrence $D(j, b) = D(j - 1, b) + D(j, b - w_j)$, with exact integers of polynomial bit length. For an allocated array, let h_e be the logarithm base two of a pair-list length. The inherited weighted entropy estimate, with the improved padding constant, is

$$\sum_e t_e h_e \le \left(\frac{\sqrt{\sum_e t_e^2}}{r} + o(1)\right) K(M, u), \qquad K(M, u) = \frac{\pi\sqrt{2}}{\ln 2}\frac{M}{\sqrt{u}}, \tag{1}$$

for any fixed nonnegative weights t_e. This bound is uniform even for small or zero pair budgets; it follows from a common generating-function parameter, not an invalid large-budget approximation on each edge separately.

3.1 Reconstructing the existing 16-group cached bound

The asymmetric supplied schedule has $u_0 \sim \lambda/81$ and $u_d \sim \lambda/(108\,4^{d-1})$ for $d \geq 1$, with $M_d \leq (1+o(1))s/8^d$. The full-mode polynomial-space sum therefore has coefficient

$$\frac{\pi\sqrt{15/8}}{\ln 2}(9+\sqrt{3}) = 66.6051434076\ldots.$$

For the prior cached method, partition the 16 groups into sizes $5, 5, 6$ and put $\omega_S = \log_2 7$. The star-cache, naive block construction, multiplication, and boundary storage factors are respectively

$$a_0 = \frac{\sqrt{30}}{16}, \quad a_B = \frac{\sqrt{150}}{16}, \quad b_S = \frac{\sqrt{120+50(\omega_S-2)^2}}{16}, \quad j_B = \frac{\sqrt{120}}{16}.$$

The root K coefficient is $9\pi\sqrt{2}/\ln 2$. Since the fully charged tail is smaller than the root multiplication term, the existing composition has time coefficient $44.5375718550\ldots$ and space coefficient $39.4959495337\ldots$. This also shows why a new polynomial-space coefficient of 50 or 60, by itself, would not improve the unrestricted-time frontier.

3.2 Why the replacement preserves the matched bound

For a block U of k groups, the old construction enumerates all its incident directed pair axes, whose number is $2k(r-k)+k(k-1)$. If $U = A \sqcup B$, with $|A| = a$ and $|B| = b$, the new merge has only its $2k(r-k)$ boundary axes and the $2ab$ shared axes. Since

$$2ab \leq k(k-1) \qquad (a, b \geq 1),$$

even the straightforward merge has no larger entropy certificate than that block's old exhaustive construction. Every intermediate subset has its incident axes contained in those of the final block. Fixed r means there are only a constant number of merges. A faster matrix multiplication backend further lowers the merging bound; it also lowers the final three-block multiplication bound relative to Strassen. The star-cache cost and recursive child oracles are unchanged.

For the storage comparison, require each top-level block to contain at most $r/2$ groups. Its intermediate subsets then have no larger boundary-table envelope, since $2j(r-j)$ increases for $j \leq r/2$. Thus, for *every fixed admissible schedule and three-block partition satisfying this block-size condition*, the new local time envelope is no larger and storage retains the old boundary-table envelope. All balanced partitions used here satisfy the condition. The old builder may be retained for more imbalanced partitions. Recurrence monotonicity carries the replacement through the recursion. It need not give a strict leading gain on a schedule whose child cache cost is already the bottleneck. The explicit schedule proved next makes the gain strict. This pointwise replacement statement is stronger and more informative than claiming that one arbitrary old parameter choice was globally optimal.

A matched ablation of the two changes. Use exactly the new strict root/tail schedule below for all three variants. The earlier flat block builder with Strassen has coefficient 38.4844959057. Giving that same flat builder the faster exponent-12/5 backend reduces the coefficient to 37.5461655198; its root table-building term is $(3/4)K_0^*$. Replacing flat construction by the binary tree reduces it further to 35.3545880584, a 5.84% coefficient decrease from that already strengthened control. These are analytical upper-bound comparisons at matched parameters. The improvements are not inferred from a search for an optimal old schedule.

4 A stronger cached contraction by recursive table construction

This section gives a new coefficient certificate for the normalized tensor interface used by the reviewed packets. Let $s \geq 2$, $\lambda = \lceil \log_2 s \rceil$, and let $\mathcal{L}$ be the input-description length. The input is a bipartite Boolean tensor network with at most s wires, maximum degree at most $\lceil s^{9/10} \rceil$, and entries in

$$(\mathbb{Z}/2^{P_0}\mathbb{Z})[V]/(V^{\ell}), \qquad P_0, \ell \leq s.$$

The permitted initial entry routines use $\mathrm{poly}(s + \mathcal{L})2^{O(s^{9/10})}$ time and polynomial space. Constants in these hypotheses are fixed. The unchanged deformation, integrality, Hermite recovery, recursive factorization, and creation-precision conventions of the source manuscript are imported. The asymmetric balancing and slope-four Cauchy separation are imported from the incoming packets. Thus the following is a relative algorithmic refinement of those exact identities; it is not an independent certification of the upstream ETH claim.

Theorem 4.1 (Cached coefficient certificate)**.** *Under this interface, the construction below runs in*

$$(s + \mathcal{L})^{O(1)}2^{(C_T + o(1))s/\sqrt{\lambda}} \quad \textit{time}, \qquad (s + \mathcal{L})^{O(1)}2^{(C_S + o(1))s/\sqrt{\lambda}} \quad \textit{space},$$

where

$$C_T = \frac{\pi\sqrt{24339}}{20\ln 2} = 35.354588058401\ldots, \qquad C_S = \frac{\pi\sqrt{915}}{4\ln 2} = 34.274803005023\ldots.$$

In particular, 35.36 *and* 34.28 *are valid respective coefficients for all sufficiently large s. A simpler rounded statement uses* 36 *for time and* 35 *for space. This variant uses subexponential, rather than polynomial, space.*

The new steps are a coordinate-and-CRT bridge to fast field matrix multiplication, and building each group-block tensor by a binary contraction tree. The latter prevents table construction from becoming the bottleneck when the matrix multiplication backend is improved. The construction retains exact per-pair weight allocation and pays for every recursively evaluated group-function entry.

4.1 The exact pair-index interface

At a retained interpolation node, write M for the current wire count, u for the taper length, and $D = 2^{a-i}$. Nearly lossless padding gives padded mass $P = (1 + o(1))M$ and child size at most $(2/r + o(1))M$. The slope-four modes have weights

$$w_q = a + 1 - i + 4q - 2s_2(q),$$

where s_2 is binary popcount. There are at most $(2 + o(1))M/(r^2 D)$ modes of any given index q for each directed pair $g \neq h$. Each such mode has exactly one source group and one target group. The sharp Hermite precision is

$$p_t \leq (6 + o(1))MD/u.$$

Enumerate arrays of exact pair budgets (b_{gh}) with $\sum b_{gh} < p_t$. Since r is a fixed integer, there are polynomially many such arrays. For one array, let $\mathcal{I}_{gh}$ be the finite set of mode-exponent tuples of exact weight b_{gh} on that pair. An empty set makes the array contribute zero. Exact-weight lists can be counted, ranked, unranked, and enumerated with polynomial resources per index by the integer recurrence

$$D(j, b) = D(j - 1, b) + D(j, b - w_j), \qquad D(0, 0) = 1,$$

with negative budgets assigned zero. These counts have polynomial bit length. The contribution of the array is exactly a graphical tensor contraction

$$\sum_{(\xi_{gh})\in\prod_{g\neq h}\mathcal{I}_{gh}}\prod_{g=1}^{r}Z_g((\xi_e)_{e\ni g}).$$

An entry of Z_g is obtained by contracting the prescribed child network. It is not a unit-cost oracle. Put $h_e=\log_2|\mathcal{I}_e|$; ceilings for binary indexing change all subsequent bounds by only $O_r(1)$.

For any fixed nonnegative weights z_e, the weighted-partition calculation gives

$$\sum_e z_e h_e \le \left(\frac{\sqrt{\sum_e z_e^2}}{r}+o(1)\right)K(M,u),\qquad K(M,u)=\frac{\pi\sqrt{2}}{\ln 2}\frac{M}{\sqrt{u}}. \tag{2}$$

To check the constant, use $\sum_{q\ge0}-\ln(1-e^{-\theta(1+4q-2s_2(q))})=\pi^2/(24\theta)+O(\log^2(2/\theta))$. For pair e, choose generating-product parameter θ/z_e and multiply its logarithmic bound by z_e. Summing gives leading terms

$$\theta p_t+\frac{\pi^2}{24\theta}\frac{(2+o(1))M}{r^2D}\sum_e z_e^2.$$

Optimizing proves (2). This argument is valid for zero or small individual pair budgets: no separate large-budget approximation for an individual list is used. The additive buffer below makes the error uniform at every active depth.

4.2 Fast matrix multiplication over the actual coefficient ring

The field-matrix result needed here is only the established strict inequality $\omega_{\mathbb{Q}}<12/5$. For example, Alman, Duan, Vassilevska Williams, Xu, Xu, and Zhou, *More Asymmetry Yields Faster Matrix Multiplication* (arXiv:2404.16349), prove a stronger exponent bound [9]. We use 12/5 with fixed slack; no current-record assertion is needed.

Lemma 4.2 (Integer-coordinate bridge)**.** *Let R be a free module of polynomial rank over $\mathbb{Z}/2^p\mathbb{Z}$, with a specified polynomial-size integer multiplication table, where p is polynomial in s. Assume the logarithms of the matrix dimensions are polynomial in s. Rectangular matrix multiplication over R can be performed in*

$$\mathrm{poly}(s+\mathcal{L})\,d_1d_2d_3\big(\min(d_1,d_2,d_3)\big)^{-3/5}$$

bit operations, with storage bounded by a polynomial factor times the input/output matrix sizes. This applies to the source ring $R=(\mathbb{Z}/2^p\mathbb{Z})[U,V]/(f(U),V^\ell)$.

Proof. Fix once and for all a finite rational bilinear matrix-multiplication algorithm with exponent strictly below 12/5, available from the quoted strict field bound. Clear its finitely many constant denominators. For every prime outside the resulting fixed exceptional set, the same bilinear identities hold over that prime field. Recursing on this fixed base algorithm, computing one recursive product at a time and accumulating the output blocks, gives quadratic field-element storage and exponent at most 12/5. Its coefficients and base dimension are constants, however large, independent of s and the matrix entries.

Lift every coordinate of an input entry to its integer representative in $[0,2^p)$. For each pair of module coordinates, compute the corresponding ordinary integer matrix product. Each unreduced integer entry is at most $d_2(2^p-1)^2$, whose bit length is at most $2p+\lceil\log_2 d_2\rceil+1$, hence polynomial in s. Put $B=2p+\lceil\log_2 d_2\rceil+1$. Compute the product modulo the first B nonexceptional odd

primes; their product exceeds 2^B and hence the reconstruction bound. These primes have polynomial numerical magnitude in B and can be found with a polynomial-size sieve. Field arithmetic and CRT recovery then take polynomial work per arithmetic or table-entry operation. Recover the nonnegative integer products, apply the specified integer structure constants, and reduce modulo 2^p. Polynomially many coordinate pairs suffice. The quotient-ring multiplication table for $f(U)$ and V^ℓ is obtained by ordinary polynomial reduction and has polynomial size and bit length.

For rectangular matrices, split into square blocks of side comparable to $\min(d_1, d_2, d_3)$ and pad by at most a constant factor. The number of block products is $O(d_1 d_2 d_3 / \min(d_j)^3)$; the square cost proves the formula. Process one product and one CRT prime at a time. Store the input/output arrays, accumulated residue information of polynomial bit length per cell, and one recursive square product. This gives the stated storage. At no point is two, a nilpotent, or a general nonunit inverted in R. □

In terms of the three index-length logarithms H_1, H_2, H_3, the matrix product exponent is

$$H_1 + H_2 + H_3 - \frac{3}{5}\min(H_1, H_2, H_3). \tag{3}$$

It is the maximum of the three weighted sums assigning weights $1, 1, 2/5$ to these classes. If their respective directed-pair cardinalities are v_1, v_2, v_3, (2) consequently bounds it by

$$\left(\frac{\sqrt{v_1 + v_2 + v_3 - (21/25)\min(v_1, v_2, v_3)}}{r} + o(1)\right) K(M, u). \tag{4}$$

The discounted class in this worst-case bound is the smallest cardinality class. This remains an upper bound for arbitrary unbalanced actual index lengths.

4.3 Build block tables recursively

Cache every Z_g on all its incident indices first. There are $2(r-1)$ incident directed pairs, so the number of recursively evaluated child entries is at most

$$2^{(a_r + o(1))K(M,u)}, \qquad a_r = \frac{\sqrt{2(r-1)}}{r}.$$

Polynomial factors include the r groups and all exact weight arrays.

For a subset B of groups, define its table T_B on the directed indices between B and its complement by summing the product of its cached group functions over all indices internal to B. For disjoint A, B, the tables T_A, T_B share precisely the directed indices between A and B. Their other index sets are disjoint: one joins A to the outside of $A \cup B$, the other joins B to that outside. Thus $T_{A \cup B}$ is an ordinary rectangular matrix product after reshaping.

If $|A| = a$, $|B| = b$, and $k = a + b$, its three index classes have cardinalities

$$2a(r-k), \qquad 2ab, \qquad 2b(r-k). \tag{5}$$

Use (4) for this merge. Recursively split every block into two parts whose sizes differ by at most one. The stored output table for size k has $2k(r-k)$ boundary directed pairs. The cached singleton tables are the base cases, so there is no repeated recursive child call hidden in table construction.

Finally divide all groups into three nonempty blocks with sizes as equal as possible. Contract their tables as a triangle using the same rectangular matrix multiplication. For block sizes k_1, k_2, k_3, the three classes have cardinalities $2k_1k_2, 2k_1k_3, 2k_2k_3$. These operations give the following exact cardinality ledger. A listed cost is squared before division by r^2; its square root divided by r is the coefficient on K.

r	block sizes	cache	largest merge	triangle	largest boundary
8	$3,3,2$	14	$766/25$	$798/25$	30
12	$4,4,4$	22	$1632/25$	$1728/25$	64

For $r=8$, a size-three merge splits $1+2$: its classes are $10,4,20$, giving $34-(21/25)4=766/25$. The triangle classes are $18,12,12$, giving $42-(21/25)12=798/25$. For $r=12$, a size-four merge splits $2+2$: its classes are $32,8,32$, giving $72-(21/25)8=1632/25$. The triangle has three classes of size 32, giving $96-(21/25)32=1728/25$. Smaller merges are less costly. Hence the triangle dominates construction in both cases.

Set

$$b_8=\frac{\sqrt{798/25}}{8},\quad j_8=\frac{\sqrt{30}}{8};\qquad b_{12}=\frac{\sqrt{1728/25}}{12},\quad j_{12}=\frac{2}{3}.$$

Per call the rigorous time recurrence, suppressing polynomial factors, is

$$T_d\le 2^{(a_{r_d}+o(1))K_d}T_{d+1}+2^{(b_{r_d}+o(1))K_d}, \tag{6}$$

and local stored tables use $2^{(j_{r_d}+o(1))K_d}$ cells. Matrix reshaping can be done by sorting records on their indices. Sequential block processing, external sorting, and fixed-base recursive multiplication add polynomial overhead in the index and record lengths, rather than assuming unit-cost access to exponentially large integers or tables.

4.4 A rational schedule with strict margins

Choose

$$\begin{aligned} r_0&=8, & u_0&=\lfloor\lambda/61\rfloor,\\ r_d&=12\quad(d\ge1), & u_d&=\left\lfloor\frac{125\lambda}{13176\,6^{d-1}}\right\rfloor\quad(d\ge1),\\ b&=\lceil 6\log_2\lambda\rceil, & a_d&=\sum_{j=0}^{d}(u_j+b),\\ H&=\lceil s/\lambda^{3/4}\rceil. \end{aligned}$$

The exact identity

$$\frac{1}{61}+\frac{125}{13176}\frac{1}{1-1/6}=\frac{1}{36}$$

gives $a_d\le\lambda/36+O((\log\lambda)^2)$ on a logarithmic-depth path. The threshold $s/\lambda^{3/4}$ is deliberate. Using s/λ with this same factor-six taper decay would not justify a uniformly growing u_d.

Here is the bootstrap without a circular depth premise. Consider hypothetical active depths up to $3\log_5\lambda/4+O(1)$ with $M>H$. Then

$$u_d\ge\lambda^{\eta+o(1)},\qquad \eta=1-\frac{3\ln 6}{4\ln 5}=0.1650\ldots>0.$$

At the root, with $Q_0=8^{a_0}$, the dominant symmetric packing term has exponent

$$Q_0^2s^{9/10}=s^{6/61+9/10+o(1)}=s^{609/610+o(1)}.$$

Below the root, all retained vertices have degree at most $s^{9/10+o(1)}$, while the fresh opposite side has degree at most $2^{a_{d-1}+1}$. The asymmetric conditions are $Q_dR_d\,\mathrm{polylog}(s)=o(M)$, $Q_d^2C_d\,\mathrm{polylog}(s)=o(M)$, and $2^{a_d}Q_d^2=o(M)$. The corresponding exponents are bounded by

$$\frac{9}{10}+\frac{\log_2 12}{36}<1,\qquad \frac{2\log_2 12+1}{36}<1.$$

For an entirely rational certificate, $12^{100} < 2^{359}$ proves $\log_2 12 < 359/100$, leaving a margin of $1/3600$ in the first inequality. New packet degrees have exponent at most $1/36 + o(1) < 9/10$. All gaps absorb logarithmic factors, the buffers, and $s/M < \lambda^{3/4}$.

Nearly lossless padding therefore holds uniformly throughout the hypothetical range. The root children have size $(1/4 + o(1))M$, and tail children $(1/6 + o(1))M$, in particular at most $M/5$ in the tail eventually. This proves the proposed logarithmic-depth bound and closes the induction. The sharper uniform shrink ratios give

$$M_0 \le s, \qquad M_d \le (1 + o(1))^d \frac{s}{4\,6^{d-1}} \quad (d \ge 1).$$

The buffer also yields $D \ge 2^{a_{d-1}+b+1}$. Inherited precision is at most $s\lambda^{O(1)}2^{a_{d-1}}$; the explicit source estimate $O(s\lambda^2 2^{a_{d-1}})$ suffices. Relative to MD/u, it is $O(\lambda^{15/4}2^{-b}) = o(1)$. The local evaluation loss is lower order as well. Thus the precision and entropy premises above hold uniformly. All arithmetic sizes stay polynomial in s.

4.5 Sum the entire recursion, including all child oracles

Let $\kappa = \pi\sqrt{2}/\ln 2$ and define

$$K_0^* = \kappa\sqrt{61}, \qquad K_1^* = \frac{\kappa}{4}\sqrt{\frac{13176}{125}}, \qquad \gamma = \frac{1}{\sqrt{6}}.$$

On the scale $s/\sqrt{\lambda}$, the root K-coefficient is K_0^*, and tail coefficients are at most $(1 + o(1))K_1^*\gamma^{d-1}$. Products of the polynomial factors over logarithmically many depths contribute only $O(\lambda \log \lambda)$ bits to the time exponent. From (6), the full coefficient is at most

$$\max\left\{b_8 K_0^*, \quad a_8 K_0^* + K_1^* \max\left(b_{12}, \frac{a_{12}}{1-\gamma}\right)\right\}. \tag{7}$$

The second inner maximum is b_{12}, and the two outer candidates are respectively

$$35.354588058401\ldots, \qquad 34.812362370059\ldots.$$

The first therefore proves the claimed time coefficient. Ancestor tables occupy a sum of geometrically declining exponential sizes, not a product. Their largest coefficients are

$$j_8 K_0^* = 34.274803005023\ldots, \qquad j_{12}K_1^* = 10.967936961607\ldots,$$

which proves the claimed space coefficient. Temporary CRT and coordinate data cost only polynomial factors per cell.

Leaves take $2^{O(s/\lambda^{3/4})}$ local work, initial entry routines take $2^{O(s^{9/10})}$, and newly created packet routines take $\exp(O(2^{a_d}\log s)) = \exp(s^{1/36+o(1)})$. All are lower order on the displayed scale, including after paying for the ancestor cached-child enumerations. Immutable creation-time descriptions prevent an unaccounted recursive composition of entry routines. This completes the proof of Theorem 4.1.

Interpretation and limits. The new theorem improves a certified leading coefficient in an exponential upper bound. It does not improve the global $n \log\log n/\sqrt{\log n}$ order or the fixed-density $n/\sqrt{\log n}$ order after the imported SAT reductions. The numerical s is normalized wire count, not SAT variable count. The field algorithm is a theoretical backend; the accompanying finite code checks its coordinate/CRT interface, not a production implementation of a Coppersmith–Winograd-type algorithm. No practical SAT speedup, new formal proof, or global optimality over all possible schedules is claimed.

5 Linear-work evaluation of full-crossing Cauchy features

Let $L = 2^a$, $R = 2^i$, $0 \le i < a$, and $N = L/2$. Define

$$s_j = (j \bmod R) + 2R\lfloor j/R \rfloor, \qquad 0 \le j < N, \quad T = t + 2^{a+1}\zeta, \quad v_2(t) = i, \quad -L \le t < L,$$

where ζ is integral in an unramified extension of $\mathbb{Q}_2$. Put $q_b = \lfloor b/R \rfloor$ and $E_b = q_b + v_2(q_b!) = 2q_b - \mathrm{popcount}(q_b)$. For a fixed port j, the reviewed normalized factors are

$$f_b(j) = 2^{-E_b} \frac{T}{T + s_j - s_b} \prod_{h<b} \frac{s_j - s_h}{T + s_j - s_h}, \qquad g_b(j) = 2^{-E_b} \frac{T}{T + s_b - s_j} \prod_{h<b} \frac{s_h - s_j}{T + s_h - s_j}.$$

Both factors vanish when $j < b$. Writing $C(X) = \prod_{q=-L}^{L-1}(X - q)$ and $A = C(T)/(C'(0)T)$, the target/source pair used in the kernel is

$$(A 2^{2E_b} f_b(j),\ g_b(j)).$$

This section changes only how these formulas are evaluated. Their algebraic identity and the global contraction reductions remain the reviewed baseline.

Streaming recurrence. Set $P_0^+(j) = P_0^-(j) = 1$ and update

$$P_{b+1}^{\pm}(j) = P_b^{\pm}(j) \frac{\pm(s_j - s_b)}{T \pm (s_j - s_b)}.$$

Then

$$f_b(j) = 2^{-E_b} P_b^+(j) \frac{T}{T + s_j - s_b}, \qquad g_b(j) = 2^{-E_b} P_b^-(j) \frac{T}{T - s_j + s_b}.$$

The update is stopped at $b = j$; all subsequent features are zero. This handles zero products explicitly instead of assigning a finite valuation to zero.

Integrality. For two slots in the same orientation, a nonzero difference $d = s_j - s_h$ has either $v_2(d) < i$ or $v_2(d) \ge i + 1$. Therefore $v_2(T \pm d) = \min\{i, v_2(d)\}$, and both $d/(T + d)$ and $T/(T + d)$ are integral. Among the indices $h < b$ with $h \equiv j \pmod R$, there are $c \ge q_b$ indices, and their quotient indices are consecutive integers starting at zero. Their contribution to $v_2(P_b^{\pm}(j))$ is

$$c + v_2\big(q_j(q_j - 1) \cdots (q_j - c + 1)\big) \ \ge\ c + v_2(c!) \ \ge\ E_b.$$

The first inequality follows because the falling factorial is divisible by $c!$. All other factors have nonnegative valuation, as does $T/(T \pm (s_j - s_b))$. Thus the final exponent after removing E_b is nonnegative. This argument applies for $b \le j$; the remaining features are identically zero.

Fixed precision. Store each nonzero prefix as a pair (v, u) representing $2^v u$, with the unit u reduced modulo 2^p and v stored exactly. Factor every numerator and denominator before modular reduction. Invert only the unit part of a denominator. When emitting a feature, subtract the known E_b from the exact valuation and multiply the stored unit by the resulting nonnegative power of two. No division of a previously reduced modular residue occurs. A unit approximation modulo 2^p remains sufficient after multiplication by any nonnegative power of two, so the method needs no extra precision. The prefactor is treated identically after cancelling T:

$$A = \prod_{\substack{-L \le q < L \\ q \ne 0}} \frac{T - q}{-q}.$$

A zero numerator gives $A = 0$ exactly. For nonzero ζ, factorial cancellation gives $v_2(A) = a + 1 - i + v_2(\zeta) \ge 0$.

Cost and compatibility. For P requested ports and $B \le N$ modes, separate evaluation of every product costs $O(L + PB^2)$ ring operations and unit inversions. The recurrence costs $O(L + PB)$ of each: the common prefactor costs $O(L)$, and every emitted feature and prefix update takes constant work. These are arithmetic counts at the same precision and in the same coefficient ring. Per-port streaming uses $O(1)$ auxiliary ring elements and integer counters; storing all outputs uses $O(PB)$ elements. Features of weight $a + 1 - i + 2E_b \ge p$ may be dropped exactly, since their targets vanish modulo 2^p. The weights are nondecreasing, so dropping the tail does not interrupt a later retained prefix. The result composes with the full-orientation Cauchy modes and any of the reviewed packing or precision schedules. It improves a local polynomial factor; it does not establish a change in the global subexponential exponent or an end-to-end SAT runtime speedup.

6 Executable evidence and reproducibility

The code consists of local exact kernels and finite proof checks. It does not invoke a SAT solver and does not implement the full asymptotic matrix multiplication algorithm. The fast-backend theorem imports an established algebraic algorithm; the executable CRT checks test the ring interface and reconstruction independently of that asymptotic backend.

6.1 Exact checks

The prefix verifier uses an independent direct product control in the same modular ring, and optionally loads the unchanged supplied rational implementation from the pinned incoming archive. The recorded normal and optimized-Python runs agree. They check:

- 69 parameter cases and 8,564 target/source feature values against the direct modular control;
- 3,124 feature values against the unchanged rational baseline;
- 2,112 reconstructed crossing-matrix entries against direct evaluation;
- 11 zero-prefactor fixtures, invalid parameter rejection, and a deliberately incorrect valuation normalization that the checker detects.

These checks include non-prime-subring interpolation values in the unramified quadratic ring $(\mathbb{Z}/2^p\mathbb{Z})[U]/(U^2 + U + 1)$. The proof applies to arbitrary finite unramified extensions, but that general extension-degree implementation is not included.

An independent cached-table checker verifies 48 small networks, 144 binary merges, 14,448 table entries, and 48 three-block contractions. It also verifies 72 rectangular CRT products and 486 product entries, both over $\mathbb{Z}/2^p\mathbb{Z}$ and over a ring with $U^2 + U + 1 = 0$ and $V^2 = 0$, for $p \in \{1, 3, 7\}$. Two deliberate corruptions are detected. This independently exercises nilpotents, negative structure constants, index bookkeeping, and exact reconstruction. Schedule arithmetic checks validate the rational budget, strict exponent margins, recurrence constants, and merge counts. Finite fixtures support the implementation and bookkeeping; the all-size theorem rests on the proofs and their explicitly named imports.

6.2 Paired local benchmarks

Both timed methods compute the same normalized features using the same modular ring operations and the same requested precision. The control recomputes each prefix product from the beginning.

The replacement retains each prefix's valuation and unit. Both include common-prefactor construction and materialize their output vectors; no precomputation is carried across runs. Each case uses three interleaved repeats, and every timed output is checked for exact equality. Four final ports of the orientation are sampled, so almost all requested features are active.

Retained modes	Precision	Direct median (s)	Prefix median (s)	Ratio
8	128	0.000677	0.000415	1.63
32	128	0.011840	0.001914	6.19
128	128	0.208508	0.007927	26.30
256	128	0.905541	0.017406	52.02
512	128	3.851839	0.037781	101.95
512	256	5.301607	0.041325	128.29

The full precision, packet, valuation, timing, and platform metadata is in `results/prefix_benchmark.json`; the CSV preserves the tabulated values. In the 512-mode, 128-bit case the largest retained mode weight is 58, so the improvement does not come from declaring high-weight modes zero. The multiplication count drops from 2,093,066 to 20,410 and the unit-inversion count from 1,044,489 to 8,161. These arithmetic counts are more portable than wall-clock timings on shared hardware. The tiny first timing is particularly sensitive to fixed overhead.

This benchmark is intentionally a substantial basis-construction workload. It is not a representative SAT benchmark suite. The measured ratio must not be presented as a 102× end-to-end SAT acceleration. The kernel's contribution to a future complete solver depends on how much time that solver spends generating features rather than enumerating tuples, evaluating tensor entries, or multiplying cached tables.

6.3 Reproduction commands

From the extracted package root, run:

```
python3 code/verify_all.py
python3 -O code/verify_all.py
python3 code/benchmark_prefix.py --repeats 3 \
  --output results/reproduced_benchmark.json \
  --csv results/reproduced_benchmark.csv
python3 code/build_report.py
```

Only the Python standard library is required for the exact checks and benchmarks. The report build requires pdfLaTeX and the standard LaTeX packages named in its preamble. The optional rational-baseline check requires the pinned source member; its SHA-256 and archive identity are recorded, and `README.txt` gives the command. The core package checks run offline without that source. The manifest verifies release bytes; re-running benchmarks creates separate files and does not change the recorded measurements.

7 SAT consequences, integration, and remaining questions

The strongest density-sensitive order claimed by the reviewed reductions remains

$$\mathrm{poly}_k(n,m)\,2^{O_k\left(\frac{n}{\sqrt{\log n}}\min\{\log(2+m/n),\log\log n\}\right)}. \tag{8}$$

At fixed density this is $\mathrm{poly}_k(n,m)2^{O_k(n/\sqrt{\log n})}$. At $m = \Theta(n \log\log n)$, the numerator can be $n \log\log\log n$. Neither new component changes these logarithmic orders. The cached variant uses subexponential space; it must not inherit a polynomial-space label from the underlying streamed algorithm.

The full construction is conditional only in the explicit, ordinary sense of importing the source framework and established algebraic results. Its new table construction, ring reduction, and schedule do not assume the unprovided mode-summation oracle from the stronger earlier conjectures. Still, the review did not certify the complete ETH claim from first principles, compile its formal development, or produce a full solver. The corpus-level trust status should remain unchanged until those larger tasks are done.

7.1 Integration plan

Keep the immutable tensor-description and creation-precision conventions. At each call, use the improved pair balancing and full-orientation Cauchy modes. Form exact-weight pair lists, pay for each cached group value through the existing child contraction, and then build the three boundary tables via the specified binary trees. Use the exact coordinate/CRT wrapper for the fast field backend. The implementation can choose Strassen or direct multiplication at small sizes without affecting the asymptotic theorem.

For the working prototype, replace the repeated-product feature evaluator with `PrefixKernel.features`. If a generalized extension-ring implementation already exists, the same valuation/unit recurrence can be ported into that arithmetic layer. Do not obtain normalized features by dividing a reduced residue by a power of two; that would lose information. Track the power separately from the outset.

An appropriate integration location is a new research directory beneath `Computability/SubexponentialSAT/Research`. Add a row describing the exact theorem and the local benchmark scope. Keep the new report, scripts, recorded results, and source manifest together. This archive does not modify the remote repository.

7.2 Further research questions

1. **Can cached contraction exploit the observed pair entropies?** The bound uses a worst-case Cauchy–Schwarz estimate for edge classes. For a particular exact-weight array, its actual list lengths may favor a different split tree or a rectangular multiplication routine. An optimizer over the fixed group set is feasible; proving a uniformly smaller coefficient requires analyzing all budget distributions.

2. **Can a smaller-space algorithm obtain the same time coefficient?** Recomputation, separator schedules, or batched boundary slices may interpolate between polynomial space and full caching. The cost of reconstructing a group value must remain explicit. A memory reduction without paying its repeated oracle calls would be spurious.

3. **Can actual sparse port sets beat the padded Smith profile?** The slope-four invariant factors are optimal for the complete padded crossing block under invertible integral basis changes. They do not constrain a submatrix with many missing real ports. An instance-sensitive pivot order could reduce finite mode lists; the cost of finding it must be included.

4. **Can the $O(L)$ common prefactor and remaining inversions be reduced?** The prefix recurrence removes quadratic repetition. Product trees, denominator reuse, or batch unit

inversion could improve further constants, with a controlled memory tradeoff. A fair benchmark should keep both precision and arithmetic representation fixed.

5. **Can the logarithmic order be improved?** The existing linear mode-weight growth and retained-level packing budget still yield $M/\sqrt{\log s}$. A faster summation theorem or a smaller valid SAT-to-network encoding would be needed for a different order. The proposed $n/\log n$ interfaces remain open.

6. **Can the pipeline become an executable, formally checked counter?** The next substantial engineering step is a complete exact contraction engine with canonical precision tags, entry oracles, interpolation, group caching, and certified cost accounting. This would enable meaningful SAT benchmarks and provide clear targets for Lean or Rocq verification.

8 Evidence ledger and limitations

The package separates three types of evidence. Mathematical proofs establish the new all-size statements relative to their named imports. Deterministic exact checks exercise the finite identities and implementations. Timings measure only the supplied local kernel under the recorded environment. None of these categories substitutes for the others.

The source pin freezes the review while the repository continues to evolve. A later commit may contain another improvement; the recorded path history and digests make this comparison reproducible. The report does not claim a lower bound on any old algorithm's actual running time, a globally optimal coefficient, or a priority result beyond the reviewed corpus. The very small positive packing margins imply potentially enormous asymptotic crossover sizes. The prefix benchmark is the concrete practical result that can be reproduced immediately.

References

[1] Vladimir Reshetnikov, *ProveIt: Sub-exponential 3-SAT project index*, initial snapshot `ab2fbc9e03d5c3d0ad6dec8686960bfe498b0b06`, closing snapshot `474ddba696a5d385c091c23fb2995f69edd13e00`. https://github.com/VladimirReshetnikov/ProveIt/blob/ab2fbc9e03d5c3d0ad6dec8686960bfe498b0b06/Computability/SubexponentialSAT/README.md.

[2] Vladimir Reshetnikov, *ProveIt incoming research archives*, arrival `3e57705cf14ee9ac99c0ac35cfff751dcf70b396`. https://github.com/VladimirReshetnikov/ProveIt/tree/ab2fbc9e03d5c3d0ad6dec8686960bfe498b0b06/docs/incoming. The source manifest records each ZIP's Git blob and SHA-256 digest.

[3] OpenAI, *The Exponential Time Hypothesis is False*, October 25, 2026; pinned source `7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a`. https://github.com/openai/math/tree/7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026. In particular, `build/quantitative-contraction.tex`, Git blob `51ba54efc29f51bbc0426f848003030da484f354`, and the adjacent reduction source.

[4] *Asymmetric packet balancing for subexponential SAT*, October 26, 2026. Incoming archive `eth_asymmetric_packet_balancing_2026-10-26.zip`; Git blob `4313cf09e210adb48564296893e45c4d3633b903`.

[5] *Full Crossing-Block Cauchy Modes: An improvement beyond the reviewed ProveIt ETH proposals and a faster option using more memory*, October 26, 2026. Incoming archive `eth_joint_cauchy_2026-10-26.zip`; Git blob `009f8b85966ef630792223aef685594b24aab520`. The local finite factorization, full-block Smith profile, and weighted-entropy bound are imported from its report.

[6] The seven previously placed refinement packets, October 26, 2026, listed in Section 2; their report sources and hashes are recorded in `provenance/source_files.json`. https://github.com/VladimirReshetnikov/ProveIt/tree/ab2fbc9e03d5c3d0ad6dec8686960bfe498b0b06/Computability/SubexponentialSAT/Research.

[7] Claude (Anthropic), *A $2^{o(n)}$ time algorithm for 3-SAT*, October 26, 2026, 49 pages; supplied at `Research/claude-2o-n-3sat/eth.pdf` in the pinned project. The reported Lean development is not independently checked here.

[8] Vladimir Reshetnikov, ProveIt second-round placement, `6d3adc45061535ebe6fa7cd3d9b37dff7dcfc22a`, reviewed at `474ddba696a5d385c091c23fb2995f69edd13e00`. https://github.com/VladimirReshetnikov/ProveIt/tree/474ddba696a5d385c091c23fb2995f69edd13e00/Computability/SubexponentialSAT/Research.

[9] J. Alman, R. Duan, V. Vassilevska Williams, Y. Xu, Z. Xu, and R. Zhou, *More Asymmetry Yields Faster Matrix Multiplication*, arXiv:2404.16349v1, April 25, 2024. https://arxiv.org/html/2404.16349v1. Only the conservative consequence $\omega < 12/5$ and the field-algebraic formulation are used; no claim about the latest matrix multiplication record is needed.